

MiniRust Language Specification

Version: v1.0

This document defines the formal specification of the MiniRust programming language. MiniRust is designed as a strongly, statically typed procedural programming language intended for pedagogical use in compiler design courses. It combines C-style program structures with Rust-style syntax, safety, and type rules.

SECTION 1: INTRODUCTION

MiniRust is a statically typed, procedural programming language combining block-structured scoping, explicit type notation, and a safe subset of Rust syntax. It enforces default immutability for variables to encourage safe data flows, while supporting mutability when explicitly declared. MiniRust provides custom composite record types via structures, static homogeneous arrays, basic type inference for local variables, and a standard control flow set including conditional statements, loops with `break/continue` constructs, and a `switch` statement. The language does not support global variables, generic types, or pointer arithmetic, keeping implementation clean and accessible for students building their first compiler.

SECTION 2: PROGRAM STRUCTURE

A MiniRust program is represented by a single source file consisting of a sequence of structure definitions followed by function definitions. Struct and function definitions are simply sequenced without separators. Struct definitions must appear at the top level before any function definitions to ensure they are visible. All struct definitions and function definitions are mutually visible throughout the entire program (file scope), meaning a struct or function can reference other structs or functions declared after it in the source file without requiring forward declarations. Global variables are not supported; all variables must be declared within function parameters or function bodies.

The entry point of every MiniRust program is a function named `main`. The `main` function must accept no arguments and must specify a return type of `void`.

A function is defined using the `fn` keyword, followed by the function name identifier, a parameter list enclosed in parentheses, an optional arrow `->` followed by a return type name, and a mandatory statement block representing the function body.

- The return type can be a primitive type, a static array type, or a user-defined struct type.
- Inside the parameter list, individual parameters are separated by commas. The list of parameters can be empty (zero parameters).
- Each parameter consists of an identifier, a colon, and its type name.
- Function parameters are passed by value and behave as local, immutable variables inside the function body.

Short Examples:

- *A simple program structure:*

```
struct Point {  
    x: i32,  
    y: i32  
}
```

```
fn add(a: i32, b: i32) -> i32 {
    return a + b;
}

fn main() {
    let p = Point { x: 5, y: 10 };
    println_i32(add(p.x, p.y));
}
```

SECTION 3: LEXICAL STRUCTURE

MiniRust source text is represented as a sequence of UTF-8 characters. The lexer translates this stream into tokens, ignoring whitespace and comments.

Comments

- **Single-line comments:** Begin with two consecutive forward slashes `//` and terminate at the end of the current line.
- **Multi-line comments:** Begin with a forward slash followed by an asterisk `/*`, and terminate with an asterisk followed by a forward slash `*/`. Multi-line comments do not nest.

Identifiers

An identifier is a sequence of ASCII alphanumeric characters and underscores. It must begin with an ASCII letter (either uppercase or lowercase) or an underscore. Identifiers are case-sensitive and must not match any keyword.

Keywords

The following keyword tokens are reserved and cannot be used as identifiers: `let`, `mut`, `fn`, `struct`, `if`, `else`, `while`, `for`, `in`, `break`, `continue`, `switch`, `case`, `default`, `return`, `i32`, `f32`, `bool`, `void`, `string`, `true`, `false`.

Literals

- **Integer Literals:** Consist of a sequence of one or more decimal digits. Matches type `i32`.
- **Floating-Point Literals:** Evaluates to type `f32`. It can be represented in either:
 - Standard fractional notation: a sequence of one or more decimal digits containing a single decimal point and another sequence of one or more decimal digits.
 - Scientific exponent notation: a sequence of decimal digits optionally containing a decimal point, followed by the character 'e' or 'E', an optional sign '+' or '-', and another sequence of one or more decimal digits.
- **Boolean Literals:** Represented by the keywords `true` and `false`. Matches type `bool`.
- **String Literals:** Consist of a sequence of characters enclosed in double quotation marks. They must support escape sequences, specifically: `\n` (representing a newline character), `\t` (representing a horizontal tab character), `\"` (representing a literal double quote character inside the string), and `\\` (representing a literal backslash character). Matches type `string`.

Operators and Punctuators

MiniRust defines distinct operators and punctuation characters:

- Arithmetic: `+`, `-`, `*`, `/`, `%`

- Comparison: `==` , `!=` , `<` , `<=` , `>` , `>=`
- Logical: `&&` , `||` , `!`
- Assignment: `=`
- Punctuation: `;` , `,` , `:` , `->` , `..` , `.` , `(` , `)` , `{` , `}` , `[` , `]`

Short Examples:

```
// Single-line comment example
let mut offset_x = -150;
let factor = 3.14e-5;    // scientific float literal
let flag = true;
let name = "Point \"A\"\\tValue\\Path"; // string with escaped quotes and backslashes
/* This is a
   multi-line comment */
```

SECTION 4: DATA TYPES

MiniRust supports primitive types and user-defined composite types.

Primitive Types

- `i32` : A signed 32-bit integer type.
- `f32` : A 32-bit single-precision floating-point type.
- `bool` : A boolean type representing logical truth values.
- `string` : A sequence of character data, used primarily as literals in standard library print calls.
- `void` : Represents the absence of a value, used as the return type for functions that do not return data.

Static Array Types

An array represents a fixed-size homogeneous collection of elements.

- **Array Types:** Specified by enclosing the element type and a positive integer literal representing its size in square brackets, separated by a semicolon (e.g., `[type; size]`).
- **Array Indexing:** Written as `array_expression[index_expression]` . Here:
 - `array_expression` represents any expression that evaluates to an array type.
 - `index_expression` represents any expression that evaluates to type `i32` .
- **Array Literals:** Array elements are enclosed in square brackets and separated by commas. The list of elements can be empty.

Structure Types

Structures are user-defined record types composed of named fields.

- **Declaration:** Declared using the `struct` keyword, followed by the struct name identifier, and a body block enclosed in curly braces. Inside the block is a list of field declarations separated by commas. The list must contain at least one field. Each field declaration consists of a field name identifier, a colon, and its type.
- **Instantiation:** starts with the struct name identifier, followed by a list of field initializers enclosed in curly braces. Field initializers are separated by commas. The list of initializers must contain at least one initializer. Each initializer consists of the field name identifier, a colon, and the initial value expression.

- **Field Access:** Members of a structure are accessed using dot notation:

`struct_expression.field_name` . Here:

- `struct_expression` represents any expression that evaluates to a user-defined structure type.
- `field_name` represents a valid field identifier declared in that structure's definition.

Short Examples:

```
let my_array: [i32; 3] = [10, 20, 30];
let first = my_array[0];

struct Employee {
    id: i32,
    salary: f32
}
let emp = Employee { id: 1001, salary: 2500.50 };
let cash = emp.salary;
```

SECTION 5: VARIABLE DECLARATION

Variables in MiniRust are declared using the `let` keyword. By default, variables are immutable and cannot be reassigned once initialized. Declaring a variable with mutability requires the keyword modifier combination `let mut` .

MiniRust variable declarations are strictly single-variable. A statement starts with `let` or `let mut` and declares exactly one variable. Multiple comma-separated declarations (e.g. `let x, y = ...`) are forbidden.

Syntactic Structure and Components

A variable declaration follows one of three syntax patterns:

1. **Explicitly Typed with Initializer:** `let name: Type = expression;` (or `let mut name: Type = expression;` for mutable variables).
2. **Explicitly Typed and Uninitialized:** `let mut name: Type;` (this pattern is only allowed for mutable variables; immutable variables must be initialized immediately).
3. **Implicitly Typed (Type Inference):** `let name = expression;` (or `let mut name = expression;` for mutable variables).

In these syntax patterns, each component is defined as follows:

- **name** : A valid identifier representing the variable's name. It must follow the rules for identifiers (defined in SECTION 3) and cannot conflict with reserved keywords.
- **Type** : A valid type name representing the variable's data type. This must be either a primitive type (such as `i32` , `f32` , `bool` , `string`), a static array type (e.g. `[i32; 10]`), or a user-defined struct type name (e.g. `Point`).
- **expression** : Any valid MiniRust expression (defined in SECTION 8) that evaluates to a value. The evaluated value's type must match the declared `Type` (subject to implicit coercion rules defined in SECTION 8).

Boundaries of Type Inference

Type inference in MiniRust is strictly local and bottom-up. The compiler computes the type of the right-hand side initializer `expression` and assigns that type to the declared variable `name`. Type inference does NOT perform bidirectional type reconstruction, constraint solving, or operand type inference. Function parameters and uninitialized variables must always be explicitly typed.

Short Examples:

```
let limit: i32 = 100;    // Explicitly typed, immutable
let mut count = 0;      // Inferred as i32 (bottom-up), mutable
let mut ratio: f32;     // Explicitly typed, mutable, uninitialized
let msg = "Processing"; // Inferred as string, immutable
```

SECTION 6: STATEMENTS & CONTROL FLOW

Statements execute actions but do not evaluate to values.

Blocks

A block consists of a sequence of statements and variable declarations enclosed in curly braces `{` and `}`. A block creates a new lexical scope; any variables declared within it are not visible outside of the block. Elements inside a block are sequenced without separators.

Expression Statement

Any expression followed by a semicolon `;` forms a statement.

Conditionals (If-else)

An if-else conditional branches execution. It starts with the `if` keyword, followed by a condition expression (which must evaluate to `bool`), followed by a mandatory block representing the "then" branch. This block can optionally be followed by the `else` keyword, which in turn must be followed by either another block representing the "else" branch or a nested `if` statement.

Switch

A switch statement starts with the keyword `switch`. Immediately following the keyword is a controlling expression (which must evaluate to `i32` or `bool`) whose value determines which alternative will be selected. The controlling expression is followed by a **body block** delimited by an opening curly brace `{` and a closing curly brace `}`.

- The body block contains one or more case clauses and an optional default clause **in any order**.
- A case clause starts with the `case` keyword, followed by a literal constant, a colon, and then a **sequence of zero or more statements**.
- The optional default clause starts with the `default` keyword, a colon, and a **sequence of zero or more statements**.
- The controlling expression is evaluated **once**, before any case is considered.
- Case clauses are examined **in order**; the first clause whose literal equals the expression's value is selected.
- Execution of the selected case's statements continues **until** a terminating statement (`break` , `return` or `continue`) is encountered **or** the end of the switch block is reached.

- Because there is **implicit fall-through**, a `break` (or another terminating statement) is normally placed at the end of each case to prevent execution from flowing into the subsequent case.
- The optional default clause is treated like any other case that matches when no other case matches.
- All statements within the switch share the same lexical scope introduced by the surrounding `{ ... }`; variables declared in one case are visible to subsequent cases unless a new block is introduced.

Loops

- **While Loop:** Starts with the `while` keyword, followed by a condition expression (which must evaluate to `bool`), followed by a mandatory block representing the loop body.
- **For Loop:** Starts with the `for` keyword, followed by an identifier representing the loop variable, the `in` keyword, a start expression, the range operator `..`, an end expression, followed by a mandatory block representing the loop body. Both start and end expressions must evaluate to `i32`. The loop variable is implicitly declared in the loop block scope as an immutable `i32` local to the loop body, and the end value is exclusive.
- **Break & Continue:** The keyword `break` or `continue` followed by a semicolon `;`. `break;` immediately terminates the innermost enclosing loop. `continue;` immediately skips the remaining statements in the current loop iteration and moves to the loop control check.

Return Statement

Exits the current function. It starts with the `return` keyword, followed by an optional expression, followed by a semicolon `;`. The expression type must match the declared return type of the enclosing function, or be omitted if the return type is `void`.

Short Examples:

```
if count < 10 {
    count = count + 1;
} else {
    count = 0;
}

switch count {
    case 0:
        println_string("Zero");
        break;
    case 1:
        println_string("One");
        break;
    default:
        println_string("Other");
}

for i in 0..10 {
    if i == 5 {
        break;
    }
    continue;
}
```

```
return; // void return
```

SECTION 7: STANDARD LIBRARY

MiniRust provides predefined standard output functions in the global scope. These functions print values to the console:

- `print_i32(val: i32) -> void` and `println_i32(val: i32) -> void`
- `print_f32(val: f32) -> void` and `println_f32(val: f32) -> void`
- `print_bool(val: bool) -> void` and `println_bool(val: bool) -> void`
- `print_string(val: string) -> void` and `println_string(val: string) -> void`
- `println() -> void` (prints a newline)

Short Examples:

```
print_string("Result: ");
print_i32(42);
println(); // prints a newline
println_bool(false);
```

SECTION 8: EXPRESSIONS & OPERATOR PRECEDENCE

Expressions evaluate to a value of a specific type.

Assignment Expression

The assignment operator `=` is a right-associative expression that evaluates to `void`. It updates the storage location of a mutable target LValue (an identifier, array indexing, or struct field access) with the evaluated value of the right-hand expression. Because it evaluates to `void`, assignment expressions cannot be chained (e.g. `let x = (y = 5);` is illegal as `x` would receive type `void`).

Operator Precedence and Associativity

The precedence of operators is defined from highest (binds tightest) to lowest (binds loosest) as follows:

1. **Primary Group (Left-to-Right)**: Parentheses `()`, Array Indexing `[]`, Member Access `.`, Function Call `f(...)`
 - Function call arguments inside the parentheses are separated by commas. The list of arguments can be empty (zero arguments).
2. **Unary Group (Right-to-Left)**: Unary negative sign `-`, logical negation `!`
3. **Multiplicative Group (Left-to-Right)**: Multiplication `*`, Division `/`, Modulo `%`
4. **Additive Group (Left-to-Right)**: Addition `+`, Subtraction `-`
5. **Relational Group (Non-associative)**: `<`, `<=`, `>`, `>=` (Chaining comparisons like `a < b < c` without parentheses is a syntax error)
6. **Equality Group (Non-associative)**: `==`, `!=` (Chaining comparisons like `a == b != c` without parentheses is a syntax error)
7. **Logical AND (Left-to-Right)**: `&&`
8. **Logical OR (Left-to-Right)**: `||`

9. Assignment (Right-to-Left): =

Type Coercion

Implicit coercion is supported only from type `i32` to type `f32` in binary arithmetic operations (promoting the integer operand to a float) and during assignments where the target has type `f32` and the expression evaluates to `i32`. All other operations require exact type matching.

Short Examples:

```
let mut x = 1.0;
x = 2; // Inferred as float, assigned an integer (implicit coercion)
let res = (10 + 20) * 3; // parentheses override additive precedence
```

SECTION 9: CODE EXAMPLES

Example 1: Valid MiniRust Program

This program demonstrates correct declaration, structures, function definitions, loop structures with loop control, switch statements, implicit type coercion, and standard library print calls.

```
struct Point {
    x: i32,
    y: i32
}

struct Circle {
    center: Point,
    radius: f32
}

fn compute_double_area(c: Circle) -> f32 {
    let pi = 3.14159;
    let radius_sq = c.radius * c.radius;
    let area = pi * radius_sq;
    return area * 2; // integer 2 implicitly coerced to f32
}

fn run_switch_demo(val: i32) {
    switch val {
        case 0:
            println_string("Value is Zero");
            break;
        case 1:
            println_string("Value is One");
            break;
        default:
            println_string("Value is Greater than One");
    }
}
```

```

fn main() {
    let mut p = Point { x: 10, y: 20 };
    p.x = 15; // valid assignment to mutable struct field

    let c = Circle {
        center: p,
        radius: 5.0
    };

    let double_area = compute_double_area(c);
    print_string("Double Area: ");
    println_f32(double_area);

    let mut counter = 0;
    while counter < 3 {
        run_switch_demo(counter);
        counter = counter + 1;
    }

    for i in 0..5 {
        if i == 2 {
            continue;
        }
        print_i32(i);
    }
    println();
}

```

Example 2: Invalid MiniRust Program

This program highlights several compiler-time errors that must be caught during lexical, syntax, or semantic analysis phases.

```

struct Point {
    x: i32,
    y: i32
}

fn add_points(p1: Point, p2: Point) -> Point {
    // Error: Parameters are immutable, reassignment is illegal
    p1 = Point { x: 0, y: 0 };
    return p1;
}

fn main() {
    let p = Point { x: 5, y: 10 };

    // Error 1: Assigning to an immutable variable
    p = Point { x: 6, y: 11 };

    // Error 2: Field access on an immutable variable cannot be assigned to

```

```
p.x = 20;

// Error 3: Loop controls (break or continue) outside loop context
break;

// Error 4: Switch type mismatch (case value must match switch expr type)
let check = true;
switch check {
  case 1:
    println_string("True"); // Error: Case is i32, expr is bool
    break;
  default:
    println_string("False");
}

// Error 5: Type mismatch in assignment (no automatic f32 to i32 conversion)
let mut val: i32 = 10;
val = 5.5;
}
```

SECTION 10: CHANGE LOG

- First release under v1.0.